



AI4Math讨论班
第二集 07-
强化学习

强化学习

AI4Math 讨论班第二季-07

晚星

求真书院

2025-11-22

新闻环节

Google 最新的旗舰大语言模型 [Gemini 3 Pro](#) 于 2025 年 11 月 19 日正式发布，在多项性能指标上力压同期最强的 GPT-5.1 和 Grok-4.1。官方更是直接称其为「最先进的推理模型」。

这是 Google DeepMind 自两年前 Bard 模型的惨败后再次取回大模型 SOTA 的位置。



Gemini 3 pro 最核心的能力包括

- 显著更强的数学和代码等推理能力
- 支持文本、图像、视频、音频全模态
- 更强的提示与指令遵循能力
- 长任务规划与执行能力
- 工具使用与智能体

虽然邪恶的闭源传统使得我们对它的技术方案一无所知，但是从这里我们也可一窥工业界大模型的发展方向。

详情可参考[相关报道](#)。

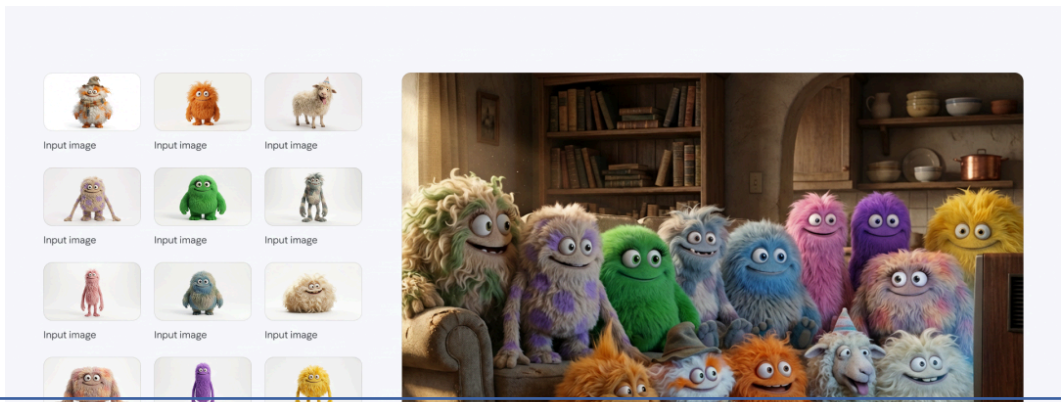
Benchmark	Description		Gemini 3 Pro	Gemini 2.5 Pro	Claude Sonnet 4.5	GPT-5.1
Humanity's Last Exam	Academic reasoning	No tools	37.5%	21.6%	13.7%	26.5%
		With search and code execution	45.8%	—	—	—
ARC-AGI-2	Visual reasoning puzzles	ARC Prize Verified	31.1%	4.9%	13.6%	17.6%
GPQA Diamond	Scientific knowledge	No tools	91.9%	86.4%	83.4%	88.1%
AIME 2025	Mathematics	No tools	95.0%	88.0%	87.0%	94.0%
		With code execution	100%	—	100%	—
MathArena Apex	Challenging Math Contest problems		23.4%	0.5%	1.6%	1.0%
MMMU-Pro	Multimodal understanding and reasoning		81.0%	68.0%	68.0%	76.0%
ScreenSpot-Pro	Screen understanding		72.7%	11.4%	36.2%	3.5%
CharXiv Reasoning	Information synthesis from complex charts		81.4%	69.6%	68.5%	69.5%
OmniDocBench 1.5	OCR	Overall Edit Distance, lower is better	0.115	0.145	0.145	0.147
Video-MMMU	Knowledge acquisition from videos		87.6%	83.6%	77.8%	80.4%
LiveCodeBench Pro	Competitive coding problems from Codeforces, ICPC, and IOI	Elo Rating, higher is better	2,439	1,775	1,418	2,243
Terminal-Bench 2.0	Agentic terminal coding	Terminus-2 agent	54.2%	32.6%	42.8%	47.6%
SWE-Bench Verified	Agentic coding	Single attempt	76.2%	59.6%	77.2%	76.3%
t2-bench	Agentic tool use		85.4%	54.9%	84.7%	80.2%
Vending-Bench 2	Long-horizon agentic tasks	Net worth (mean), higher is better	\$5,478.16	\$573.64	\$3,838.74	\$1,473.43
FACTS Benchmark Suite	Held out internal grounding, parametric, MM, and search retrieval benchmarks		70.5%	63.4%	50.4%	50.8%
SimpleQA Verified	Parametric knowledge		72.1%	54.5%	29.3%	34.9%
MMMLU	Multilingual Q&A		91.8%	89.5%	89.1%	91.0%
Global PIQA	Commonsense reasoning across 100 Languages and Cultures		93.4%	91.5%	90.1%	90.9%
MRCR v2 (8-needle)	Long context performance	128k (average)	77.0%	58.0%	47.1%	61.6%
		1M (pointwise)	26.3%	16.4%	not supported	not supported

For details on our evaluation methodology please see deepmind.google/models/evals-methodology/gemini-3-pro

就在 Gemini 3 pro 发布之后仅一天，Google 的代表性图像生成模型也迎来了一波大幅更新：[Nano Banana Pro](#) 正式发布。其关键性能进步在于：

- 支持至多 4K 分辨率图像的生成和多种长宽比
- 更细致的编辑控制选项
 - 更强的动嘴修图能力
- 完善的世界知识
- 显著更好的文本控制能力

这也标志着 Google 在图像生成领域也同时取得了一定的领先地位。



而在这一切成果的背后，其基础技术仍然是 Transformer 神经网络与基于随机梯度下降的训练流程。

经典的强化学习

在此前我们已经介绍过大语言模型的基本构成：

- 模型结构：Transformer
- 工作方式：自回归生成
- 优化方法：AdamW 等

而有了这一切之后，剩下的就只需要根据我们的最终目标构造特定的损失函数作为优化目标，并朝着这个方向训练模型即可。

这里最经典的方法也就是监督学习 (Supervised Learning, SL)，它就是参考着神经网络或者机器学习最初的想法，直接通过现有数据做函数拟合。

例如在基础的图像生成当中使用的方法基本是首先收集大量的图片 (可能带有类别或文本标注)，然后计算神经网络预测重建的图像和原数据的二范数的平方。

$$\mathcal{L}(\theta) = \mathbb{E} \|x_0 - x_{0,\theta}(x_t, t)\|^2$$

而大语言模型的预训练和监督微调阶段主要所做的事情也就是模仿训练数据的模式，它所使用的损失函数也就是交叉熵损失：

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \sum \log p_{\theta}(x_t \mid x_{<t})$$

它与强化学习的主要差异仅在损失函数的构造上。

人们普遍相信最具扩展潜力的学习方法必然建立在自主性的基础上。即一个智能体在观测和行动等与环境的交互过程当中不断地自主学习与进化。其中常常涉及到的几个概念是：

State ($\mathcal{S}$) 这是系统所有可能状态的集合。

Action ($\mathcal{A}$) 这是智能体在各个状态下所能采取的动作的集合。

Transition Function ($T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1], (s, a, s') \mapsto P(s' | s, a)$) 这一函数表征在某一个给定状态以及动作下各种可能的状态改变发生的概率。

Reward ($r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$) 表示在特定状态下采取动作时得到的反馈。

此时这一智能体通常就被表达为一个策略 (policy) 函数 $\pi(a | s)$, 预测在每个情形下选取动作的概率。



回报函数 $R(\tau) = \sum \gamma^t r_t$ ，通常针对特定任务，根据 reward 进行定义，代表某一个动作轨迹所带来的最终回报。

价值函数 $V^\pi(s) = \mathbb{E}_{a_t \sim \pi(s_t)} R(\tau)$ ，指的是在特定状态以及策略下回报函数的期望。

动作价值函数 $Q^\pi(s_0, a_0)$ ，指的是在特定状态下执行某一个动作之后，特定策略下回报函数的期望。

根据这些我们就可以得到强化学习理论当中经典的贝尔曼方程 (Bellman Equation)，它说明了一个状态的价值等于当前获得的奖励加上后续状态价值的加权平均。

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(r(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s') \right)$$

接着根据最优情形下的贝尔曼方程进行迭代，学习寻找动作价值函数的方法就被称为 Q-Learning。这是一个与现代深度学习完全不同的古典算法。其具体算法表示为

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

经典的策略梯度 REINFORCE 算法提出于 1992 年，在它之前强化学习的主流是价值函数方法，但它们很难直接表示一个复杂的随机策略。REINFORCE 的训练目标就是直接对参数化策略 $\pi_{\theta}(a | s)$ 做梯度上升。其核心公式就在于：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

它同时可以被简化为

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

此处的 G_t 代表的是从 t 时刻往后累计的回报。

直觉上 REINFORCE 所做的就是将高回报的轨迹概率提高，同时将低回报的轨迹概率降低。如此就可以训练出整体更优的策略了。

不过在当时的应用场景当中 REINFORCE 的问题就在于每一条动作轨迹回报波动很大，这导致其对于梯度的估计噪声很大且训练并不稳定。

自此之后基于 policy 做优化的强化学习算法逐步成为主流。

Actor-Critic 的思想在强化学习理论早期就已出现，而后来的神经网络则成为了标准的拟合工具用于扮演这两个角色。其中：

- Critic 学习每一个状态的价值信息
- Actor 根据 Critic 的反馈调整策略

这里额外 Critic 的意义就在于拟合 $V^\pi(s_t)$ 评估每一个状态的平均价值，并以此平衡各步动作，偏好相对更优的动作而非绝对更优。如此即可实现更好的训练稳定性。

在引入 Critic 之后我们就可以将 REINFORCE 当中的 G_t 替换为 $G_t - V_\theta(s_t)$ ，作为优势函数的估计。而精确的优势函数定义则是

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

Advantage function 的概念由此引入，这一思路也被后来的各种强化学习工作所继承。

TRPO (Trust Region Policy Optimization) 算法最早在 2015 年提出，它是深度强化学习其中的一个重量级工作。TRPO 所使用的目标函数核心项是

$$\mathbb{E} \left[\frac{\pi_{\theta}(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} \hat{A}(s, a) \right]$$

同时额外引入了可信域约束，进一步增强了稳定性，具体而言它就是：

$$\mathbb{E} \left[\text{KL} \left(\pi_{\theta_{\text{old}}}(\cdot | s) \parallel \pi_{\theta}(\cdot | s) \right) \right] \leq \delta$$

在这里 TRPO 训练使用的轨迹都由未经训练的旧策略 $\pi_{\theta_{\text{old}}}(a | s)$ 生成，这也可以更好地保持训练的稳定性。也正因此 TRPO 引入了重要性采样来修正其中的偏差，也就是使用

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

代替直接通过新策略模型计算的梯度，由此正确地估计新策略下的优势函数值。

实践上 TRPO 通常需要使用共轭梯度等方法来进行求解，实现上通常也非常复杂。

PPO (Proximal Policy Optimization) 于 2017 年发布，它是对 TRPO 算法的一次大幅简化，并取得了良好的性能表现。在此后的数年里 PPO 一直是强化学习算法的标杆。

它最主要的优化就是将 TRPO 当中复杂的可信域计算简化为一个简单的策略商截断，其目标主项就是：

$$\mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

也就是将偏离旧策略太多的 r_t 直接截断到一个固定值，并参与后续的各种计算。

PPO 属于一个在理论上并不优雅，但是工程实践当中却非常方便，能够有效防止策略变化过大导致的问题。

后来在实践当中也常常向它的损失函数当中额外添加熵或者 KL 散度两项，以鼓励更多样的探索轨迹或者保持更好的稳定性。

$$H(\pi_{\theta}(\cdot | s)) = - \sum_a \pi_{\theta}(a | s) \log \pi_{\theta}(a | s)$$

$$\begin{aligned} D_{\text{KL}}(\pi_{\theta}(\cdot | s) \parallel \pi_{\theta_{\text{old}}}(\cdot | s)) \\ = \sum_a \pi_{\theta_{\text{old}}}(a | s) \log \frac{\pi_{\theta_{\text{old}}}(a | s)}{\pi_{\theta}(a | s)} \end{aligned}$$

LLM 中的 RL

在前大语言模型时代，NLP 领域就存在许多使用 RL 算法去优化不可导指标的工作。而现今大语言模型的工作方式是自回归生成，或者称为 next-token prediction，它可以在大或小两种不同尺度上视作一个策略模型：

小尺度上：

- 状态：补全时的上文
- 动作：生成下一个 token
- 策略：就是语言模型 $\pi_{\theta}(a_t \mid a_{<t})$

与此同时在大尺度上，例如基于 LLM 构建的智能体任务当中，有

- 状态：上文内容和环境观测结果
- 动作：生成一段代码并交给程序执行
- 策略：从当前状态出发生成出下一段动作

那么由此我们就可以在不同的尺度上对大语言模型应用强化学习算法，并训练它实现我们所期望的各种功能。

接下来的问题就是如何构建强化学习算法所需要的环境 reward 信息。

RLHF (Reinforcement Learning from Human Feedback) 算法首次提出于 OpenAI 2022 年的一篇论文 [InstructGPT](#)，其中首次通过 RLHF 这一算法赋予了大语言模型强大的指令遵循能力以及执行语言领域通用任务的能力。它也是大模型发展早期最受关注的算法之一。

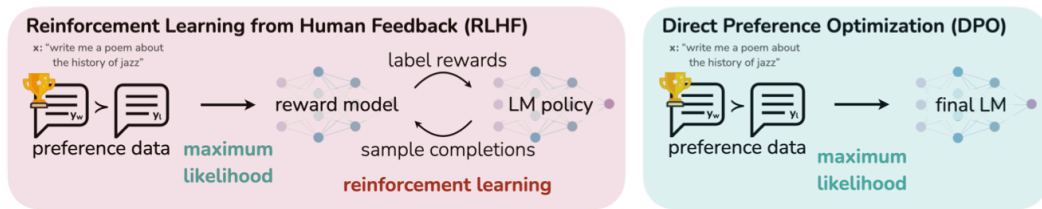
RLHF 的实现一般分三步

1. 从预训练模型出发收集大量人工标注的指令与回答数据，直接进行监督训练微调，由此得到一个还不错会听话的初始 policy。

2. 给定同一个 prompt 并从初始 policy 当中采样多条模型回答，并让人类标注者提供偏好反馈。后再训练一个 reward model 来为每一个回答预测一个标量的 reward。(此时这样的 reward model 通常就是直接初始化一个大语言模型并接上一个标量输出头)
3. 使用 PPO 算法做强化学习，同时需要额外加上许多约束防止崩溃。

实际上现代大语言模型已经基本不再直接使用 RLHF 算法，在获取到足够多的标注数据之后只需简单的 SFT 就已有足够强的指令遵循效果了。

DPO (Direct Preference Optimization) 算法首次提出于 2023 年，它是针对 RLHF 流程的一次大幅简化。



它直接从偏好数据对出发，不经过中间训练 reward model 和完整的强化学习流程，而是直接从偏好数据对 (x, y^+, y^-) 出发构建损失函数。

DPO 具体的目标函数计算公式是：

$$\mathbb{E}_{x, y^+, y^-} [(\log \pi_{\theta}(y^+ | x) - \log \pi_{\text{ref}}(y^+ | x)) - (\log \pi_{\theta}(y^- | x) - \log \pi_{\text{ref}}(y^- | x))]$$

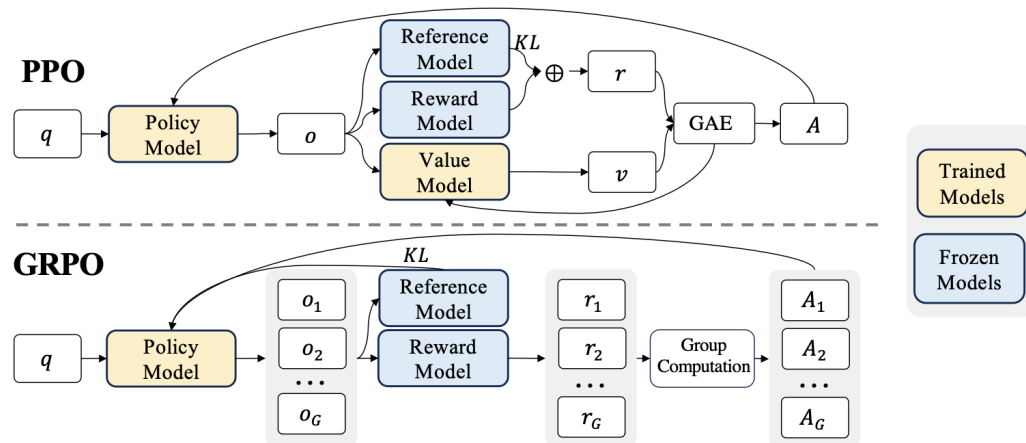
可以看到它在形式上已经和直接的监督训练差别不大了，仅仅是根据对比数据构造了一些相对损失。由此 DPO 算法在偏好对齐训练上能够做到简便、稳定，且效果上几乎不输甚至可能略优于完整的 PPO-based RLHF。

当然其实现代很多时候单纯地使用交叉熵损失做 SFT 就已经够用了。

GRPO (Group Relative Policy Optimization) 算法首次提出于 2024 年的论文 [DeepSeek-Math](#)。它在 PPO 的基础上进一步进行简化，删去了 value model，并直接使用多组回复的平均 reward 作为基准计算优势函数。如此一来优势函数就直接是

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

这对于数学、代码等简单的问题设定而言非常直接且有效。



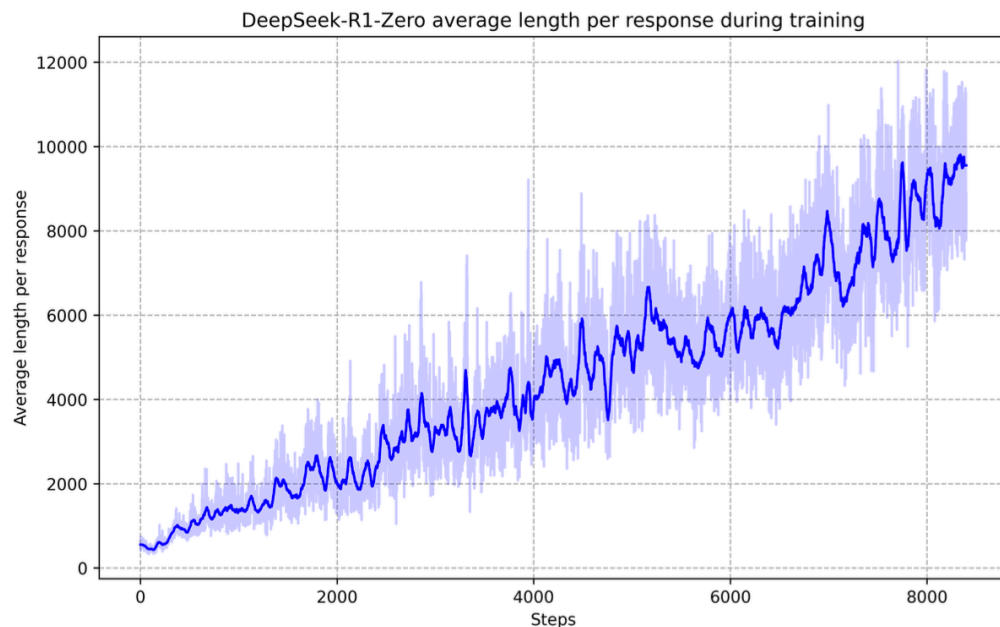
Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_{θ} by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

2025 年初左右兴起的推理模型技术就是强化学习技术在大语言模型当中的又一轮高光。而在后续的改良工作当中人们发现进一步删去 reward model 使用可验证损失，删去 reference model 以及 KL 散度项，都不会影响训练的稳定性。由此产生出的算法被统称为 RLVR (Reinforcement Learning from Verifiable Reward)。

大语言模型在针对推理问题的大量强化学习训练下自发地产生了深度思考的能力，GRPO 也在这一年里成为了强化学习算法的新基准。



$$(y^u - y^{u+1}) - \lambda \rightarrow u - y^{u+1} - \lambda$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

Dr. GRPO 是相对标准 GRPO 算法的一次有效改进，它基于对 GRPO 算法的一些关键观察并修正了其中可能存在的一些偏差，并在实验当中取得了更好的效果。

原本 GRPO 损失函数当中存在的一个关键问题在于长度偏差，因为 reward 按照 response-level 进行分配，除以 token 数量平均之后就会在 token level 产生长度偏置。具体体现在：

- 短而正确的回复每个 token 的 reward 比长而正确的更高
- 长而错误的回复每个 token 的 reward 比短而错误的更高

GRPO

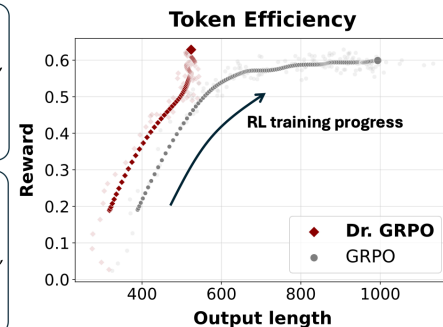
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\},$$

where $\hat{A}_{i,t} = \frac{R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})}$.

Dr. GRPO
GRPO Done Right (without bias)

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | \mathbf{q}, \mathbf{o}_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right\},$$

where $\hat{A}_{i,t} = R(\mathbf{q}, \mathbf{o}_i) - \text{mean}(\{R(\mathbf{q}, \mathbf{o}_1), \dots, R(\mathbf{q}, \mathbf{o}_G)\})$.



这会意外地引导模型偏好长而错误的回复，产生很多无用的错误成果。

Dr. GRPO 的另一项改进在于删去了优势函数当中的归一化，因为他们认为这种归一化会加强极端情况而削弱回复正确率方差较大时的情形，而后者才是最有学习价值的。

GSPO (Group Sequence Policy Optimization) 则由阿里巴巴通义实验室于 2025.7.28 首次提出，它指出了原本 GRPO 当中的另一个关键问题：它错误地使用了 PPO 式的重要性采样。

由于 TRPO 当年重要性采样所期望实现的目标是根据随机变量的分布差异修正 reward 数值，但是这里 reward 是在序列级生效的，而非 token 层级。因此 GSPO 当中将策略商的数值修改为了

$$s_i(\theta) = \exp \left(\frac{1}{|y_i|} \sum_t \log \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})} \right)$$

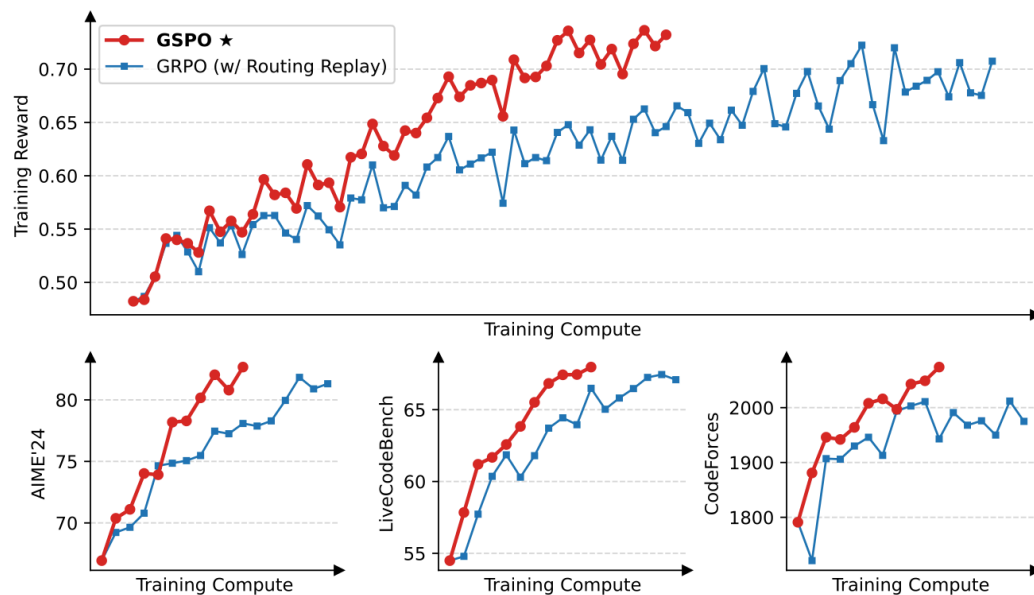


Figure 1: Training curves of a cold-start model fine-tuned from Qwen3-30B-A3B-Base. GSPO possesses remarkably higher training efficiency than GRPO.

GSPO 当中的 clipping 机制也在序列级实现。实验当中 GSPO 确实呈现出了更强的性能表现，尤其在 MoE 模型当中优势非常明显。

实际上即使深度思考能力也未必需要由深度学习得到，只需要获取相应的长思维链数据进行微调，就可以直接指导语言模型学会深度思考的工作方式，并取得显著的性能提升 (参见 DeepSeek-R1 与 Qwen3 等论文)。

如果对比参看标准监督训练和 REINFORCE, DPO 等的损失函数的话，除去 reward 放缩外，基本上可以认为 SFT 就是只有正例的 RL 算法。或者另一方面可以认为 RL 也是某种模仿训练的数据来自自身探索而非人为标注的微调手段。

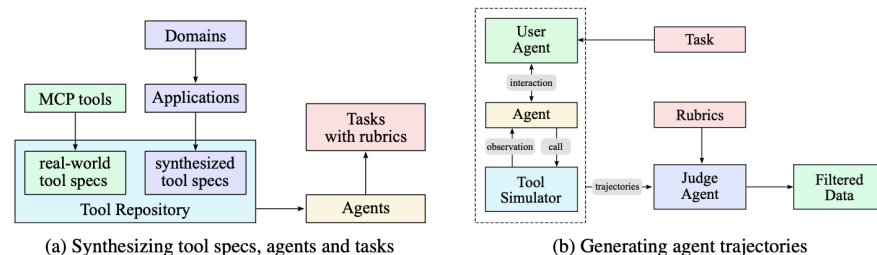
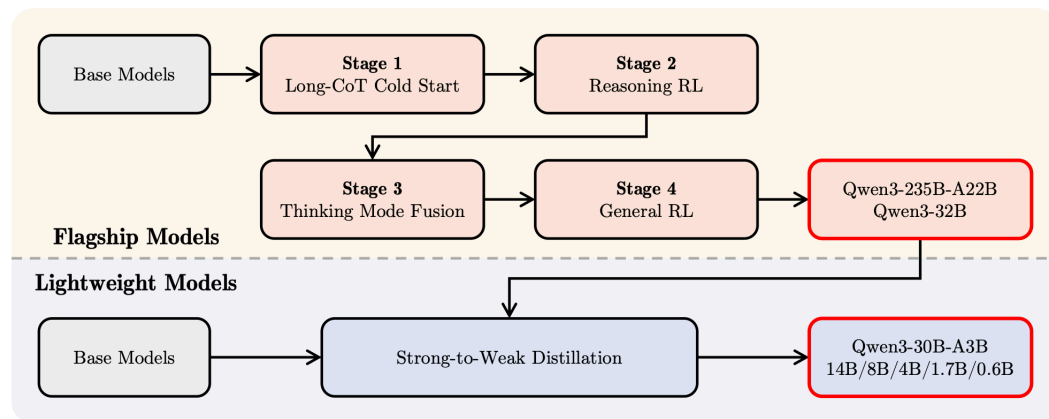


Figure 8: Data synthesis pipeline for tool use. (a) Tool specs are from both real-world tools and LLMs; agents and tasks are the generated from the tool repo. (b) Multi-agent pipeline to generate and filter trajectories with tool calling.

下回预告

本集当中我们已经完成了现代语言模型各方面技术的讲解。在下集当中我们将会讲解 VAE, GAN, Diffusion, Flow Matching 等视觉生成模型的相关情况，作为 AI 系列的补完。